

Towards Mixed-Criticality Software Architectures for Centralized HPC Platforms in Software-Defined Vehicles: A Systematic Literature Review^{*}

Lucas Mauser¹, Eva Zimmermann², Pavel Nedvědícký², Tobias
Eisenreich², Moritz Wäschle¹, and Stefan Wagner²

¹ Daimler Truck AG, Leinfelden-Echterdingen, Germany

² Technical University of Munich, Heilbronn, Germany

Abstract. Centralized electrical/electronic architectures and High-Performance Computers (HPCs) are redefining automotive software development, challenging traditional microcontroller-based approaches. Ensuring real-time, safety, and scalability in software-defined vehicles necessitates reevaluating how mixed-criticality software is integrated into centralized architectures. While existing research on automotive Software Architectures (SWAs) is relevant to the industry, it often lacks validation through systematic, empirical methods. To address this gap, we conduct a systematic literature review focusing on automotive mixed-criticality SWAs. Our goal is to provide practitioner-oriented guidelines that assist automotive software architects and developers design centralized, mixed-criticality SWAs based on a rigorous and transparent methodology. First, we set up a systematic review protocol grounded in established guidelines. Second, we apply this protocol to identify relevant studies. Third, we extract key functional domains, constraints, and enabling technologies that drive changes in automotive SWAs, thereby assessing the protocol's effectiveness. Additionally, we extract techniques, architectural patterns, and design practices for integrating mixed-criticality requirements into HPC-based SWAs, further demonstrating the protocol's applicability. Based on these insights, we propose an exemplary SWA for a microprocessor-based system-on-chip. In conclusion, this study provides a structured approach to explore and realize mixed-criticality software integration for next-generation automotive SWAs, offering valuable insights for industry and research applications.

Keywords: Software engineering · Software architecture · Software-defined vehicle · HPC platform · Mixed-criticality system · Virtualization · Systematic literature review

1 Introduction

Today's emerging technologies in automotive, such as powertrain electrification, computing-power-intensive autonomous driving, bandwidth-hungry info-

^{*} Preprint submitted to the research paper track of ECSA 2025

tainment systems, enhanced connectivity, and the resulting need for cybersecurity, challenge the component-, domain-oriented development of distributed, embedded Electrical/Electronic (E/E) architectures [8]. Within these, system properties such as busload, computing power, fault rate, modularity, and flexibility are reaching their limits [29]. The Software-Defined Vehicle (SDV) and its centralized E/E architecture promise to address these limitations as an enabler to accelerate innovation, reduce time-to-market of features, and revolutionize business models [44,10]. It will enable original equipment manufacturers to continuously develop, integrate, and deploy software faster and more flexible [34].

Traditional, component-oriented development in the automotive industry resulted in embedded and distributed E/E architectures with over 100 Electronic Control Units (ECUs) [39]. By now, there has been an evolution towards partially consolidated, domain-oriented E/E architectures [8]. To differentiate from the competition, key features have been integrated into more powerful domain controllers with the target of developing customer-perceivable application software in-house [39]. Still, the domain-oriented E/E architecture is characterized by a large number of ECUs widely distributed in the vehicle and hardware-dependent functions, often called logical SoftWare Components (SWCs) [8,29].

Today, automotive E/E architectures evolve towards centralized, zonal E/E architectures, as discussed in our previous work [29], [30], and by various researchers [21], [41], [28], [27], [8]. Centralization is based on functional consolidation into a few, powerful High-Performance Computers (HPCs). Functional decomposition as an exemplary method helps abstract a system and its functionalities to extract SWCs and allocate them to central HPCs [42]. This separation of computing and Input/Output (I/O) forms the basis for centralized E/E architectures of SDVs. Here, centralization helps overcome today's distributed system limitations such as busload, computing power, and updateability [29]. While most logic resides in the HPCs, zonal gateway controllers aggregate distributed, smart I/Os to reduce overall vehicle wiring and weight [21]. More details on different types of E/E architectures can be found in literature [8,21,45,30].

Functional consolidation within an HPC brings new challenges to its Hardware (HW) and Software (SW) Architecture (SWA). The growing demand for computing power – driven not only by functional consolidation, but also by emerging technologies such as AI-based algorithms – is driving a shift from microcontroller-based (μ C-based) to microprocessor-based (μ P-based) architectures [10]. The global development partnership AUTOSAR introduced the μ C-based platform, AUTOSAR Classic, in 2003, evolving it into a widely adopted SWA for real-time and time-critical applications [6]. The AUTOSAR Adaptive platform was introduced in 2017 to support the aforementioned paradigm shift to meet the demands of service-oriented, high-performance, μ P-based SWAs [5]. AUTOSAR Adaptive is not intended to replace AUTOSAR Classic. Rather, both platforms will coexist, depending on the specific area of application [45]. In modern vehicles, HPCs integrate consolidated μ C and μ P partitions, designed as System-on-Chip (SoC) solutions, offering advantages such as reduced external communication [10] while backbone networks approach bandwidth limitations

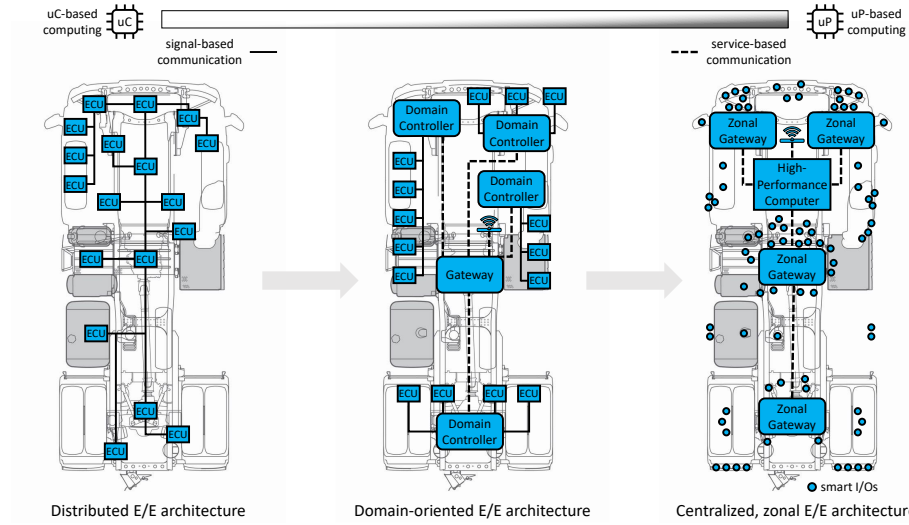


Fig. 1: Evolution of automotive E/E architectures

[46,37]. This consolidation of real-time applications with service-oriented, event-driven applications into one single SWA comprises a Mixed-Criticality System (MCS) [13]. MCSs integrate functionalities of different criticality levels. In the automotive context, these are categorized as Automotive Safety Integrity Levels (ASILs) according to the ISO 26262 (QM, ASIL A to D) [13]. Meeting the diverse requirements of real-time systems and emerging performance technologies within a centralized SWA leads to our primary research question:

RQ: *How are automotive, centralized SWAs designed to incorporate mixed-criticality requirements of SDVs?*

Addressing the research question, this paper assists SW architects and developers in consolidating mixed-criticality functions within centralized SWAs of HPC platforms, leveraging the advantages of SDVs. Focusing on technical and methodical enablers, the study bridges academic research and industry practices.

The paper is organized as follows: We review related work to contextualize our research and highlight its added value in Section 2. In Section 3, we outline the research method used for the Systematic Literature Review (SLR). The results are presented and discussed in Section 4. Addressing the threats to validity in Section 5, we conclude and provide an outlook for future research in Section 6.

2 Related Work

This section reviews related work to position our contributions. While previous studies have examined automotive SWAs, no comprehensive SLR has specifically addressed mixed-criticality architectures for centralized HPC platforms in SDVs.

Aleti et al. [2] conducted an SLR on SWA optimization methods, presenting a taxonomy to assist architects in selecting techniques. However, since their 2013 study focused on embedded systems, their scope differs from ours, as we propose a centralized, HPC-based SWA rather than optimizing existing ones. Other SLRs have a narrower focus than our goal of a universal, in-vehicle SWA for centralized HPC platforms. Banijamali et al. [9] analyzed SWAs for IoT-cloud convergence, discussing attributes, such as portability and maintainability, and Service-Oriented Architectures (SOAs) – key considerations for SDVs [30,29]. However, it does not specifically address in-vehicle architectures. Similarly, Avci et al. [7] studied big-data architectures across industries, identifying automotive use cases offloaded to the cloud rather than in-vehicle architectures, as explored in our research [29]. Beyond SLRs, several Systematic Mapping Studies (SMSs) provide broader overviews of automotive SWA research. However, their general scope limits their relevance to the specific challenges of centralized HPC platforms in SDVs. For example, Haghighatkhah et al. [19] analyzed literature on Automotive Software Engineering (ASE), identifying “system/software architecture and design, qualification testing, and reuse” as the three most frequently studied areas. Their findings highlight the critical role of SWA and design in academia and industry, as evidenced by the leading ASE research institutions spanning companies and public organizations. The review highlights research with high industrial relevance but low scientific rigor. Here, we derive and conclude, in alignment with the conclusion of Haghighatkhah et al., the need for practitioner-oriented “guidelines for selecting existing solutions, technologies, and practices” based on a rigorous SLR satisfying scientific expectations.

3 Research Method

We conducted an SLR to answer the research question to address the identified need for systematic research. This section highlights the key milestones of the three phases of planning, conducting, and reporting to ensure the key characteristics of a credible SLR according to Kitchenham et al. [22]: comprehensiveness, reproducibility, and transparency. The working products of these key milestones are documented and publicly available on GitHub and Zenodo.

3.1 Research Question

Kitchenham et al. [22] emphasize the importance of the research question as the essential part of a systematic review. Thus, we applied the PICOC (Population, Intervention, Comparison, Outcomes, Context) criteria presented within the guidelines to refine and reframe the research question. The PICOC criteria determine the research scope, ensure the consideration of different viewpoints, and help reduce bias.

P: Automotive SWAs – **I:** Centralized HPC SWA – **C:** Real-time, embedded SWA – **O:** Mixed criticality SWAs – **C:** Technological transformation in automotive industry.

With these, we substantiate our research question: *How are automotive, centralized SWAs designed to incorporate mixed-criticality requirements of SDVs?*

3.2 Review Protocol

Three performance indicators guide the selection of our SLR protocol: recall, precision, and F-measure. Recall ensures the review’s comprehensiveness by minimizing the risk of missing relevant studies, while precision ensures identified studies contribute to answering the research question. The F-measure, the harmonic mean of precision and recall, balances both metrics. [31] For this study, we selected an SLR based on a Scopus database search, which has demonstrated high precision in evaluations by Mourão et al. [31]. This approach prioritizes relevance over recall, avoiding the lower precision seen in databases like Google Scholar [31]. Threats to validity of the protocol will be discussed in Section 5.

3.3 Search String

For the Scopus search, we constructed the following search string based on the PICOC criteria:

```
TITLE-ABS-KEY ("software architecture*" AND ( vehicle OR automotive )
AND ( "mixed critical*" OR hpc OR "High*Performance Comput*" OR
"central*" OR "functional domain*" ))
```

The search string is applied to titles, abstracts, and keywords in Scopus (TITLE-ABS-KEY). We discussed, reviewed, and modified the search string in review sessions with all authors to ensure plausibility and reduce overlooks of a single author. The search string is intended to answer the research question by identifying modern, mixed-criticality automotive SWAs in recent literature.

3.4 Inclusion and Exclusion Criteria

To produce the seed set of studies from the database search, we defined the inclusion and exclusion criteria in Table 1. These include content-related criteria and quality-related criteria for achieving a certain level of quality, as discussed in the guidelines of Kitchenham et al. [22]. For a study to be included, content-related and quality-related criteria must be valid. The exclusion of non-peer-reviewed publications ensures the inclusion of high-quality studies. For the same reasons, we exclude books or sections from books, as peer review is not guaranteed. Based on the identified literature and the evolution of automotive E/E architectures described in Section 1, the SDV approach, which separates computing and I/O, gained momentum in the mid-2010s. This can also be confirmed by analyzing the search results by publication year. As a result, we exclude studies from the HW-driven and ECU-oriented era of E/E architectures before 2010. The study language is limited to English and German, in which the authors are proficient.

Table 1: Inclusion and exclusion criteria

A study is included if	A study is excluded if
IC1: the title, abstract, and keywords clearly demonstrate a contribution to the research question	EC1: the study does not meet any of the inclusion criteria
IC2: the abstract tends to answer the aspects of data extraction derived from the research question	EC2: the study is not peer-reviewed
	EC3: the study is either a book or a section from a book
	EC4: the study is published before 2010 or after December 2024
	EC5: the study is not written in English or German
	EC6: the study is not accessible

We use the tool Rayyan to efficiently document and perform the inclusion/exclusion process [33]. Each study’s inclusion and exclusion criteria are documented, with references to the short names of the corresponding criteria and a detailed rationale. The first four authors reviewed each study from the Scopus database search. This process served as a pilot to establish a common understanding and consensus on the inclusion/exclusion criteria while reducing the potential for bias from a particular author. Any initial disagreements, discussions, and resolutions have been documented in the SLR report.

3.5 Data Extraction

For data extraction of the included papers, we broke down the research question into Data Extraction Criteria (DEC) to further explore the particular aspects. The criteria facilitate data extraction and the subsequent synthesis of the study outcomes.

- DEC1 Which functional domain(s) does the study analyze and/or modify in relation to SWA changes?
- DEC2 Which constraint(s) does the study identify as drivers for SWA changes?
- DEC3 Which technologies does the study identify as enablers or catalysts for changes in the SWA?
- DEC4 How does the study technically address the integration of diverse SW requirements (real-time, non-real-time, safety-critical, etc.) within a centralized automotive SWA?
- DEC5 Which architectural patterns or design practices are proposed to systematically support mixed-criticality in centralized automotive SWAs?

To effectively divide the work of data extraction while minimizing bias, only the first author reviewed all of the literature in the seed set. Authors 2, 3, and 4 each reviewed one-third of the seed set. The extracted data were compared between the first author and the other reviewers. Agreement on the results was required, and any ambiguities have been resolved.

3.6 Data Synthesis

The data synthesis is designed based on the extraction criteria. DEC1, DEC2, and DEC3 provide context on why SWAs are facing change. For DEC1, we conducted a quantitative, deductive coding to highlight functional domains according to the domain taxonomy Vehicle Signal Specification (VSS) V5.0 by the COVESA community [15]. For DEC2 and DEC3, we conducted an explorative, qualitative open coding by extracting and grouping identified constraints and technologies from the literature without applying weightings. DEC4 and DEC5 answer the research question. We conducted a qualitative content analysis using open coding for both: DEC4 to discuss technical implementations and approaches, and DEC5 to examine architectural patterns and design practices. The results of the data extraction and synthesis are presented and discussed in the following section.

4 Results and Discussion

The search string applied in the database Scopus resulted in 97 identified studies. The inclusion/exclusion criteria application resulted in 21 studies included in the final set. The data extraction of these 21 studies is the basis for the results in this section. While we discuss each of the included studies below, we list them here in a preliminary manner to provide a central overview: [46,20,11,18,17,40,43,36,35,38,1,14,32,12,24,26,37,4,25,23,16].

4.1 Providing context

Fig. 2 illustrates the key functional domains driving change in automotive SWAs. The three most influential domains are: Advanced Driver Assistance Systems (ADASs), particularly its autonomous subcategory; powertrain, driven by electrification; and cabin, driven by infotainment. ADASs and autonomous vehicles demand high computing power to support computer vision algorithms, pushing embedded, μ C-based ECUs and their SWAs to their limits [14]. In the powertrain domain, electrification expands the design space, increasing the complexity of SW implementations and necessitating a fundamental rethinking of embedded SWAs [40]. The evolution of high-performance digital displays challenges Instrument Cluster (IC) and In-Vehicle Infotainment (IVI) systems, not only by increasing HW and SW complexity but also by requiring integration of safety-critical and non-safety-critical applications within a single, centralized SWA [20].

Such constraints of today's SWAs, identified in the literature by DEC2, and technologies as enablers and catalysts for changes in SWAs, identified in the literature by DEC3, are illustrated by the word clouds in Fig. 3.

With these extracted and synthesized data providing context, we want to focus on the research question of Section 1. To answer the research question, we set up DEC4 and DEC5, the results of which we will discuss qualitatively below.

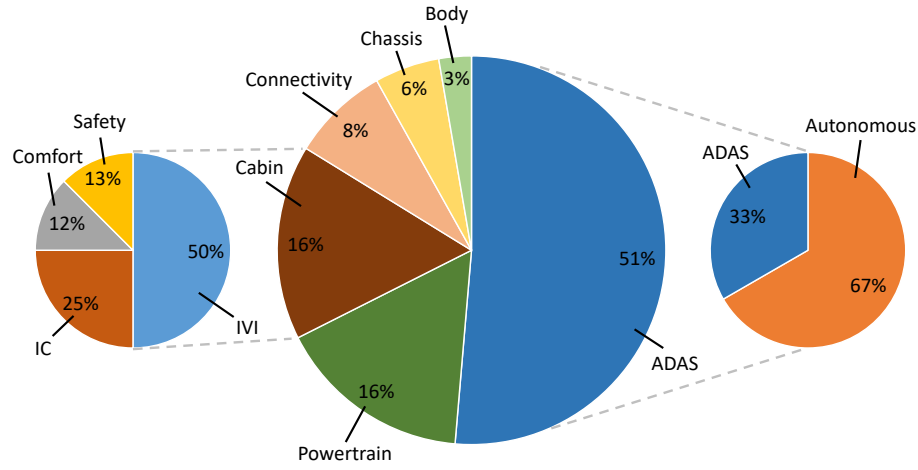


Fig. 2: DEC1 - Functional domains driving change in SWAs according to Vehicle Signal Specification (VSS) V5.0 [15]

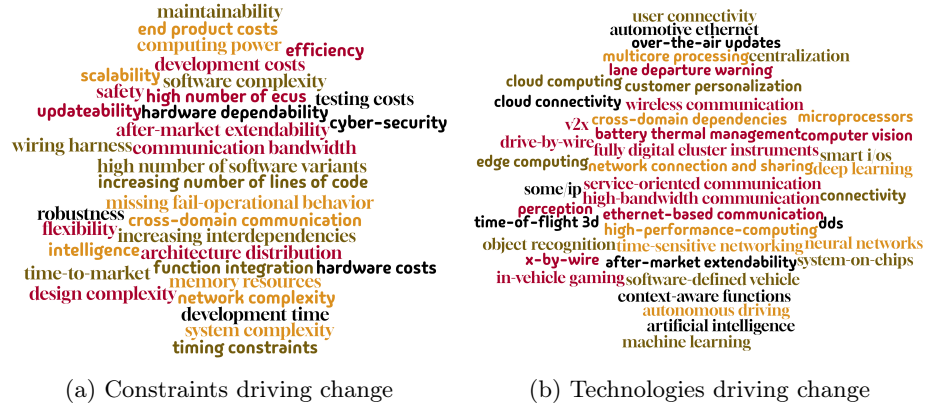


Fig. 3: Word clouds DEC2 (left) and DEC3 (right)

4.2 Answering the research question

We analyze DEC4 and DEC5 inspired by the AUTOSAR structure and its layers, starting from the HW layer, through HW-near SW layers, the operating system layer, the middleware layer as so-called basic SW in AUTOSAR Classic, to the application SW layer [6,5]. Based on the following discussion, we integrated the findings into the exemplary SWA shown in Fig. 4 illustrating key aspects for realizing mixed-criticality SWAs for HPC platforms. Details and the derivation will be elaborated in the following, based on the analysis of DEC4 and DEC5.

even bare-metal applications [20,18]. There are two commonly used types of hypervisors. Type 1 (bare-metal) hypervisors run directly on the HW, providing exclusive and static resource management, leading to higher efficiency, performance, and strongest FFI. Type 2 hypervisors run on a host OS, making HW access less efficient and introducing a risk of failure if the host OS crashes [18,4,25]. Additionally, virtualization can be classified as full or paravirtualization. Full virtualization emulates HW components, allowing unmodified guest OSs to run as if on physical HW. Paravirtualization requires guest OS modifications to interact more efficiently with the hypervisor, improving performance. The literature confirms paravirtualization offers higher performance than full virtualization, which incurs overhead due to HW emulation [18,25]. Virtualization is also seen as having “cost, reliability, availability, and adaptability” benefits while guaranteeing spatial and temporal isolation [18,36]. In summary, virtualization enables the separation of SW logic from HW and mixed OS environments as a prerequisite for SDVs [24].

OS Layer: At the OS layer, Ferraro et al. [18] propose a Real-Time OS (RTOS) for actuation and a Linux OS for performance-intensive object detection. Additionally, Robot Operating System 2 (ROS2), based on Data Distribution Service (DDS), is used in self-driving applications to enhance scalability [18]. Similarly, Lee and Wang [24] split sensing/actuation-heavy SW and processing-heavy SW. Niedballa and Reuss [32] employ an RTOS for safety- and time-critical tasks and system monitoring, while Linux OS handles computing-intensive application logic.

Middleware Layer: At the middleware layer, various methods and protocols support the coexistence of mixed-criticality OSs and applications. With increasing cross-domain communication [46,37] and dependencies [14], a key challenge is ensuring FFI as centralized HPC platforms consolidate multiple domains [46,37]. Holstein and Wietzke [20] address this risk for inter-VM communication by introducing architectural approaches such as the *clear separation approach*, *layers of interconnections*, and *minimalistic approach* with its one-way/read-only Data Containers (DCs) illustrated in Fig. 4. To maintain data integrity in mixed-criticality environments, key enablers include memory management and stack monitoring mechanisms such as isolated communication channels, access control mechanisms, signed communication data, consistency checks, redundant memory, multiple memory copies, and checksums [20,17,32]. For both communication and isolation, a Virtual Local Area Network (VLAN) enables FFI by creating virtualized network segments [18]. Also, Virtual Time-Sensitive Networking (VTSN) extends Quality-of-Service (QoS) capabilities to VMs using TSN schedulers Qav (bandwidth reservation based on criticality or priority) and Qbv (communication separation into fixed-duration periodic cycles) [18]. Although Qbv achieves lower end-to-end latency, Ferraro et al. [18] suggest combining both approaches to improve reliability and robustness. Ensuring that both functional and non-functional requirements are met at runtime is critical to meeting QoS requirements [38]. Similarly, DDS is proposed as middleware to ensure “reliable, scalable, and efficient real-time communication”

[35]. Additionally, time partitioning can enhance predictability and reliability in program execution, which Watchdog Managers (WdgMs) can monitor [40,37]. Furthermore, self-diagnostic capabilities can enhance software-sided redundancy, enabling hot-switching between redundant components to mitigate failures and maintain system continuity [1]. In general, the literature identifies a shift from statically configured, signal-oriented communication towards a more flexible, scalable, and updateable service-oriented communication [46,11,37]. This evolution is exemplified by the publisher-subscriber mechanism implemented in the automotive communication protocol Scalable service-Oriented MiddlewarE over IP (SOME/IP) [11,24], which decouples communication between applications [37]. However, this approach introduces timing uncertainties, which can be managed using the previously mentioned mechanisms, such as TSN, DDS, and other timing-aware solutions [11]. This evolution lays the foundation for centralized, data-centric Automotive Service-Oriented Architectures (ASOAs), enabling the SDV approach to update vehicles post-production in a more flexible manner and thus to evolve over time in response to new customer needs, while also reducing time to market [18,35,38,4]. Several frameworks support these advancements at the middleware layer, including: SOAFEE [20] providing real-time guarantees in SOAs by integrating cloud-native practices with automotive requirements; Time-Sensitive Autonomous Architecture (TSAA) [18] leveraging TSN protocols to support mixed-criticality SDVs; Brain Centralized Electronic and Electrical Architecture (BCEA) [26] enabling HW-SW decoupling, facilitating offloading to cloud computing; Hypervisor-based Fault tolerance approach for heterogeneous Automotive Real-time systems (HyFAR) [25] enabling mixed-criticality via virtualization technology; and AUTOSAR Adaptive as a middleware (see Sec. 1) introducing ASOAs with enhanced updateability [12,25] for centralized HPCs with high computing power and high-speed communication demand [26].

Application SW Layer: At the application SW layer, Lee and Wang [24] advocate for microservice architectures to enhance evolvability and accelerate software development and release, addressing the growing complexity of automotive software [43]. This aligns with broader recommendations in the literature to design loosely coupled SWCs that maximize HW independency, improving transferability and scalability [24,26,37]. Li et al. [26] highlight HW-SW decoupling as a critical enabler for cloud computing, facilitating computational offloading while maintaining compatibility with HPC-based architectures. To further enhance portability, isolation, and deployment flexibility, containerization is proposed as a lightweight alternative to full virtualization. In automotive HPC platforms, containers are typically deployed atop a hypervisor, combining the strong isolation of VMs with the efficiency of containers. This setup enables multiple guest OSs to run separate containerized applications, ensuring scalability, modularity, and rapid software updates while preserving FFI between mixed-criticality applications [11]. Additionally, Kugele et al. [23] propose architectural patterns, such as the *User Shadow Learning Pattern*, for integrating AI-driven personalization into automotive software architectures. These patterns enhance the application

software layer by enabling adaptive, context-aware functions that personalize vehicle behavior while maintaining safety constraints.

We conclude the section with the exemplary mixed-criticality SWA shown in Fig. 4, which incorporates these findings. While evidence emphasizes SWAs as “one of the most critical success factors for the design and development of complex software systems” [12], Fig. 4 illustrates its strong relation to the HW layer beneath. The literature often discusses and explores architecture as a whole, considering the close relationship between HW and SW from the very beginning [14,12,16]. Lee and Wang [24] even highlight HW as the design driver for automotive architectures. While the SDV approach aims to decouple SW functions from HW [24], the SWA and its requirements still depend on the HPC HW it runs on, and close co-design can help to solve this multiobjective problem [14].

The following section discusses threats to validity related to our SLR.

5 Threats to Validity

We evaluate our study’s quality by discussing the three main threats to validity in secondary software engineering studies as identified by Ampatzoglou et al. [3]: *threats to study selection*, *threats to data collection*, and *threats to research validity*.

5.1 Threats to Study Selection

Our study omits snowballing due to the balanced F-measure of Scopus searches, as detailed in Section 3.2. While some relevant papers were excluded due to access restrictions or language barriers, such as Chinese-language papers and SAE reports, the high precision of Scopus ensured a comprehensive seed set to answer our research question. We acknowledge potential publication bias and propose a search strategy for snowballing with our seed set as a basis to explore additional methodological and technical approaches in the outlook section. Additionally, industry insights may be outdated due to limited disclosure. Including gray literature or industry expert interviews could enhance understanding and identify current practices but conflicts with our focus on peer-reviewed studies. Concluding, we want to emphasize the potential threat by variability in evaluating loose inclusion criteria wordings as “clearly demonstrate” and “tend to answer”.

5.2 Threats to Data Collection

We mitigated data extraction bias by using commonly agreed-upon data extraction criteria, with each study independently reviewed by two authors. Disagreements were resolved through discussion and consensus-building. Pre-piloting of the extraction criteria ensured a common understanding. As the study neither applies snowballing nor includes gray literature, it persists the threat of publication bias related to the data collected. To still ensure validity of the related study and reduce publication bias, we considered quality-related inclusion criteria (see

Table 1) as proposed by Kitchenham et al. [22]. Furthermore, the diversity of journals and conferences from which the included publications were drawn helps to mitigate the threat of publication bias.

5.3 Threats to Research Validity

To minimize research bias, the first four authors collaboratively designed and refined the review protocol, aligning with established guidelines. The first three authors specialize in automotive SW engineering, while the fourth focuses on AI integration into SWAs. The fifth and sixth authors – experts in functional safety and SW engineering, respectively – reviewed the protocol to further reduce bias. Disagreements on protocol, data extraction, and synthesis were resolved within the core group or escalated to the reviewers. To enable repeatability, all review discussions, protocol changes, and inclusion/exclusion decisions are documented and made publicly available on GitHub and Zenodo. This study focuses on automotive SWAs and may have limited applicability to other domains due to industry-specific constraints. Future research could explore cross-industry approaches, such as avionics systems (see IMA and ARINC standards [37]), or IT and cloud-based architectures, to identify transferable best practices.

6 Conclusion and Outlook

In this paper, we systematically reviewed existing research on mixed-criticality SWAs for centralized HPC platforms in SDVs. To ensure methodological rigor and transparency in our analysis, we employed a structured research protocol grounded in established literature review practices. Through this protocol, we identified key functional domains, constraints, and enabling technologies that shape modern automotive SWAs. Furthermore, we extracted architectural patterns and design practices that support the integration of mixed-criticality requirements within centralized automotive SWAs. Our findings provide practitioner-oriented insights to aid SW architects and developers in designing HPC-based SWAs.

However, our study also revealed limitations and open challenges. The lack of publicly available industry data hinders validating recent advancements in automotive SWAs. Moreover, the transferability of best practices from other domains, such as avionics or cloud-based SWAs, requires further investigation.

Future work can build on the review protocol and expand the empirical findings with the hybrid search strategy *Scopus + BS//FS* based on its consistent high F-measure values as analyzed by Mourão et al. [31]. Our study delivers the seed set for parallel Backward Snowballing (BS) – which increases recall by identifying references cited within a study – and Forward Snowballing (FS) – which improves precision by identifying newer studies citing a given publication [31]. In addition, the study of cross-industry approaches could reveal transferable design principles for automotive applications. To capture the latest advances, the inclusion of gray literature and an empirical interview study with industry experts could uncover emerging approaches and technologies.

Acknowledgments. This research was partially supported by the German Federal Ministry of Education and Research in the project AutoDevSafeOps (01IS22087R) and the Incentive Fund of TUM Campus Heilbronn.

Disclosure of Interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Akkaya, S., et al.: A modular five-layered v-shaped architecture for autonomous vehicles. In: 2019 11th International Conference on Electrical and Electronics Engineering (ELECO). pp. 850–854. IEEE (2019), <https://doi.org/10.23919/ELECO47770.2019.8990402>
2. Aleti, A., et al.: Software architecture optimization methods: A systematic literature review. *IEEE Transactions on Software Engineering* **39**(5), 658–683 (2012), <https://doi.org/10.1109/TSE.2012.64>
3. Ampatzoglou, A., et al.: Identifying, categorizing and mitigating threats to validity in software engineering secondary studies. *Information and software technology* **106**, 201–230 (2019), <https://doi.org/10.1016/j.infsof.2018.10.006>
4. Askaripoor, H., et al.: E/e architecture synthesis: Challenges and technologies. *Electronics* **11**(4), 518 (2022), <https://doi.org/10.3390/electronics11040518>
5. AUTOSAR: Autosar adaptive platform (2024), available at: <https://www.autosar.org/standards/adaptive-platform>. Accessed: 2025-03-07
6. AUTOSAR: Autosar classic platform (2024), available at: <https://www.autosar.org/standards/classic-platform>. Accessed: 2025-03-07
7. Avci, C., et al.: Software architectures for big data: a systematic literature review. *Big Data Analytics* **5**(1), 5 (2020), <https://doi.org/10.1186/s41044-020-00045-1>
8. Bandur, V., et al.: Making the case for centralized automotive e/e architectures. *IEEE Transactions on Vehicular Technology* **70**(2), 1230–1245 (2021), <https://doi.org/10.1109/TVT.2021.3054934>
9. Banijamali, A., et al.: Software architectures of the convergence of cloud computing and the internet of things: A systematic literature review. *Information and Software Technology* **122**, 106271 (2020), <https://doi.org/10.1016/j.infsof.2020.106271>
10. Bauer, T., et al.: Reference architectures for automotive software. In: *Reference Architectures for Critical Domains: Industrial Uses and Impacts*, pp. 73–111. Springer (2022), https://doi.org/10.1007/978-3-031-16957-1_5
11. Bordoloi, U., et al.: Autonomy-driven emerging directions in software-defined vehicles. In: 2023 Design, Automation & Test in Europe Conference & Exhibition (DATE). pp. 1–6. IEEE (2023), <https://doi.org/10.23919/DATE56975.2023.10136910>
12. Bucaioni, A., et al.: Modelling centralised automotive e/e software architectures. *Advanced Engineering Informatics* **59**, 102289 (2024), <https://doi.org/10.1016/j.aei.2023.102289>
13. Cinque, M., et al.: Certify the uncertified: Towards assessment of virtualization for mixed-criticality in the automotive domain. In: 2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W). pp. 8–11. IEEE (2022), <https://doi.org/10.1109/DSN-W54100.2022.00012>

14. Collin, A., et al.: Autonomous driving systems hardware and software architecture exploration: optimizing latency and cost under safety constraints. *Systems Engineering* **23**(3), 327–337 (2020), <https://doi.org/10.1002/sys.21528>
15. COVESA: Vehicle signal specification (2025), available at: https://covesa.github.io/vehicle_signal_specification/introduction/index.html. Accessed: 2025-03-07
16. Druml, N., et al.: Time-of-flight 3d imaging for mixed-critical systems. In: 2015 IEEE 13th International Conference on Industrial Informatics (INDIN). pp. 1432–1437. IEEE (2015), <https://doi.org/10.1109/INDIN.2015.7281943>
17. El-Bayoumi, A.: An enhanced algorithm for memory systematic faults detection in multicore architectures suitable for mixed-critical automotive applications. *International Journal of Safety and Security Engineering* **10**(4), 467–474 (2020), <https://doi.org/10.18280/ijss.100405>
18. Ferraro, D., et al.: Time-sensitive autonomous architectures. *Real-Time Systems* **59**(4), 568–608 (2023), <https://doi.org/10.1007/s11241-023-09404-2>
19. Haghighatkhah, A., et al.: Automotive software engineering: A systematic mapping study. *Journal of Systems and Software* **128**, 25–55 (2017), <https://doi.org/10.1016/j.jss.2017.03.005>
20. Holstein, T., Wietzke, J.: Contradiction of separation through virtualization and inter virtual machine communication in automotive scenarios. In: Proceedings of the 2015 European Conference on Software Architecture Workshops. pp. 1–5 (2015), <https://doi.org/10.1145/2797433.2797437>
21. Kadry, H.M., et al.: Electrical architecture and in-vehicle networking: Challenges and future trends. In: 2022 IEEE International Symposium on Circuits and Systems (ISCAS). pp. 1009–1013. IEEE (2022), <https://doi.org/10.1109/ISCAS48785.2022.9937481>
22. Kitchenham, B., et al.: Guidelines for performing systematic literature reviews in software engineering. Tech. rep., Technical report, ver. 2.3 ebse (2007)
23. Kugele, S., et al.: Architectural patterns for cross-domain personalised automotive functions. In: 2020 IEEE international conference on software architecture (ICSA). pp. 191–201. IEEE (2020), <https://doi.org/10.1109/ICSA47634.2020.00026>
24. Lee, J., Wang, L.: A method for designing and analyzing automotive software architecture: A case study for an autonomous electric vehicle. In: 2021 International Conference on Computer Engineering and Artificial Intelligence (ICCEAI). pp. 20–26. IEEE (2021), <https://doi.org/10.1109/ICCEAI52939.2021.00004>
25. Lex, J., et al.: Hyfar: A hypervisor-based fault tolerance approach for heterogeneous automotive real-time systems. *Journal of Systems Architecture* **156**, 103263 (2024), <https://doi.org/10.1016/j.sysarc.2024.103263>
26. Li, Y., et al.: Key technology and standardization route for new electronic and electrical architecture of intelligent and connected vehicles. In: 2023 3rd International Conference on Electrical Engineering and Control Science (IC2ECS). pp. 323–328. IEEE (2023), <https://doi.org/10.1109/IC2ECS60824.2023.10493525>
27. Lu, S., et al.: A comparison of end-to-end architectures for connected vehicles. In: 2022 Fifth International Conference on Connected and Autonomous Driving. pp. 72–80. IEEE (2022), <https://doi.org/10.1109/MetroCAD56305.2022.00015>
28. Maier, J., Reuss, H.C.: Design of zonal e/e architectures in vehicles using a coupled approach of k-means clustering and dijkstra’s algorithm. *Energies* **16**(19), 6884 (2023), <https://doi.org/10.3390/en16196884>
29. Mauser, L., Wagner, S.: Centralization potential of automotive e/e architectures. *Journal of Systems and Software* p. 112220 (2024), <https://doi.org/10.1016/j.jss.2024.112220>

30. Mauser, L., et al.: Methodical approach for centralization evaluation of modern automotive e/e architectures. In: European Conference on Software Architecture. pp. 165–179. Springer (2022), https://doi.org/10.1007/978-3-031-36889-9_13
31. Mourão, E., et al.: On the performance of hybrid search strategies for systematic literature reviews in software engineering. *Information and software technology* **123**, 106294 (2020), <https://doi.org/10.1016/j.infsof.2020.106294>
32. Niedballa, D., Reuss, H.C.: Mpsoc-based platform for fail-operational control of an automated research vehicle. *Journal of Tongji University (Natural Science)* **50**(S1), 151–155 (2024), <https://doi.org/10.11908/j.issn.0253-374x.23717>
33. Ouzzani, M., et al.: Rayyan—a web and mobile app for systematic reviews. *Systematic reviews* **5**, 1–10 (2016), <https://doi.org/10.1186/s13643-016-0384-4>
34. Pelliccione, P., et al.: Automotive architecture framework: The experience of volvo cars. *Journal of systems architecture* **77**, 83–100 (2017), <https://doi.org/10.1016/j.sysarc.2017.02.005>
35. Püllen, D., et al.: A security process for the automotive service-oriented software architecture. *IEEE Transactions on Vehicular Technology* **73**(4), 5036–5053 (2023), <https://doi.org/10.1109/ETFA.2019.8868957>
36. Savithry, J., et al.: Design of criticality-aware scheduling for advanced driver assistance systems. In: 2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA). pp. 1407–1410. IEEE (2019), <https://doi.org/10.1109/ETFA.2019.8868957>
37. Sommer, S., et al.: Race: A centralized platform computer based architecture for automotive applications. In: 2013 IEEE International Electric Vehicle Conference (IEVC). pp. 1–6. IEEE (2013), <https://doi.org/10.1109/IEVC.2013.6681152>
38. Stähle, H., et al.: Towards the deployment of a centralized ict architecture in the automotive domain. In: 2013 2nd Mediterranean Conference on Embedded Computing. pp. 66–69. IEEE (2013), <https://doi.org/10.1109/MECO.2013.6601320>
39. Staron, M.: Automotive software architectures. Springer (2021), <https://doi.org/10.1007/978-3-030-65939-4>
40. Sundharam, S.M., et al.: Software architecture modeling of autosar-based multi-core mixed-critical electric powertrain controller. *Modelling* **2**(4), 706–727 (2021), <https://doi.org/10.3390/modelling2040038>
41. Surjekar, N.N., et al.: A case study on migrating towards functionally safe zonal architecture using mbse. In: INCOSE International Symposium. vol. 33, pp. 1403–1417. Wiley Online Library (2023), <https://doi.org/10.1002/iis2.13089>
42. Vogelsang, A.: Feature dependencies in automotive software systems: Extent, awareness, and refactoring. *Journal of Systems and Software* **160**, 110458 (2020), <https://doi.org/10.1016/j.jss.2019.110458>
43. Wallin, P., et al.: Problems and their mitigation in system and software architecting. *Information and Software Technology* **54**(7), 686–700 (2012), <https://doi.org/10.1016/j.infsof.2012.01.004>
44. Xie, G., et al.: Recent advances and future trends for automotive functional safety design methodologies. *IEEE Transactions on Industrial Informatics* **16**(9), 5629–5642 (2020), <https://doi.org/10.1109/TII.2020.2978889>
45. Zerfowski, D., Lock, A.: Functional architecture and e/e-architecture—a challenge for the automotive industry. In: 19. Internationales Stuttgarter Symposium: Automobil-und Motorentechnik. pp. 909–920. Springer (2019), https://doi.org/10.1007/978-3-658-25939-6_70
46. Zhou, X., et al.: Development of vehicle domain controller based on ethernet. In: *Journal of Physics: Conference Series*. vol. 1802, p. 022065. IOP Publishing (2021), <https://doi.org/10.1088/1742-6596/1802/2/022065>